

相关问题回答

1. CPU占用率不满但是会有任务等待

我觉得最主要的原因：

在airfogsim_algorithm.py中，scheduleComputing 函数对CPU分配策略有明确的限制：

```
def alloc_cpu_callback(computing_tasks, **kwargs):
    task_list = []
    for tasks in computing_tasks.values():
        for task in tasks:
            task_dict = task.to_dict()
            assigned_node_id = task_dict['assigned_to']
            assigned_node_info = self.entityScheduler.getNodeInfoById(env, assigned_node_id)
            task_num = appointed_fog_node_dict.get(assigned_node_id, 0)
            if assigned_node_info is None or task_num>=3:
                continue
            appointed_fog_node_dict[assigned_node_id] = task_num + 1
            task_list.append(task_dict)
```

每个计算节点最多只处理3个任务，即使节点有足够的CPU资源处理更多任务。当任务数超过这个限制时，多余的任务会处于等待状态，即使CPU资源没有被充分利用。

次要原因：

在airfogsim_algorithm.py中，任务卸载过程依赖于scheduleOffloading函数。该策略总是选择最近的节点，不考虑节点当前负载。这可能导致某些热门节点（位置优越的节点）接收大量任务，而其他节点闲置，造成整体CPU利用率不高但仍有任务等待。

2. 任务具体如何卸载

任务卸载（Task Offloading）是指将任务从源节点（通常是资源有限的设备如车辆或无人机）转移到其他计算节点（如RSU或云服务器）执行的过程。流程如下：

2.1 卸载决策（Offloading Decision）

由airfogsim_algorithm.py中scheduleOffloading函数实现，决定任务卸载到哪个节点：

1. 获取所有需要卸载的任务
2. 对每个任务，识别其源节点（task_node_id）
3. 获取源节点附近的节点（最多5个，按距离排序）
4. 选择最近的节点作为卸载目标
5. 通过taskScheduler.setTaskOffloading设置卸载目标

"最近节点优先"策略

2.2 通信资源分配 (Resource Block Allocation)

一旦决定了卸载目标，下一步是分配通信资源以传输任务数据，`scheduleCommunication`函数：

1. 获取可用资源块(RB)总数
2. 获取所有正在卸载的任务
3. 平均分配资源块给每个任务
4. 设置每个任务的通信资源分配

这些设置会被保存在环境中，以便后续的通信执行。

2.3 通信执行 (Communication Execution)

设置完卸载目标和通信资源后，**环境**的`_updateWirelessCommunication`函数会执行实际的数据传输，这个函数有3个关键步骤：

a. 分配通信资源

`_allocate_communication_RBs`函数将上一步设置的资源块分配与具体的通信链路关联，这一步确定了通信双方、通信类型（如V2I表示车辆到基础设施）和分配的资源块。

b. 计算通信速率

`_compute_communication_rate`函数计算各通信链路的数据传输速率

c. 执行通信过程

`_execute_communication`函数执行实际的数据传输，此步骤根据计算出的通信速率，在一个时间步长内传输相应大小的数据。如果任务数据完全传输完成，则标记为成功卸载；如果出现错误（如节点离开网络），则标记为失败。

2.4 任务数据传输完成后的处理

成功卸载的任务会在目标节点上等待处理。这是通过`finishOffloadingTask`方法（在`task_manager.py`中）实现的

2.5 计算资源分配

卸载完成的任务需要被分配计算资源进行处理，通过`airfogsim_algorithm.py`中`scheduleComputing`函数实现。

这个函数为每个计算任务分配CPU资源，采用简单的平均分配策略。

2.6 计算执行

环境的`_updateComputation`方法执行实际的计算，这一步使用分配的CPU资源来处理任务数据。

2.7 结果返回

如果任务需要将结果返回，scheduleReturning函数会设置返回路径。

任务在卸载过程中的状态变化

任务（Task）类中定义的生命周期状态（task_lifecycle_state属性）包括：

- to_generate：任务尚未生成
- to_offload：任务已生成，等待卸载决策
- offloading：正在传输到目标节点
- computing：正在计算中
- computed：计算完成
- returning：结果正在返回
- finished：任务完全完成

3. 生成节点和计算节点能否是同一个节点以及数量如何确定

可以是同一个节点

在airfogsim_env.py中，见92行注释

```
self.task_node_ids = []  
# list, 存储所有的task node id。  
# 可以在每个决策间隙更新，即每个车辆在不同的时候可能是fog node或task node。  
# 注意，每次生成的任务数量是按照"predictable_seconds"来预先存储的，  
# 所以可能t=0.1s时，车辆是task node，  
# t=0.2s时，车辆是fog node，  
# 但是此时还有该车辆的任务需要进行卸载或计算。
```

节点数量确定

生成节点数量

受配置文件中的设置控制，在初始化中，见环境73行：

```
self.task_node_profiles = self.config['task_profile']['task_node_profiles'] # list of dict  
# [{ 'type': 'UAV', 'max_node_num': 15}, { 'type': 'vehicle', 'max_node_num': 10}]  
self.task_node_types = [profile['type'] for profile in self.task_node_profiles]  
self.task_node_gen_poss = self.config['task_profile']['task_node_gen_poss']  
self.max_task_node_num = {}  
for profile in self.task_node_profiles:  
    self.max_task_node_num[profile['type']] = profile['max_node_num']
```

节点添加过程中的控制代码：

在_updateTraffics方法中，处理新加入的车辆（环境756-757行），处理新加入的UAV（770-771行）

可以看出：

1. 配置文件定义了每种类型节点能成为任务节点的最大数量(max_node_num)
2. 还定义了节点成为任务节点的概率(task_node_gen_oss)
3. 当新节点加入系统时，如果：
 - 该类型节点在允许的任务节点类型中
 - 随机数小于成为任务节点的概率
 - 该类型的任务节点数量未达到上限
 - 该节点不在当前任务节点列表中

则将该节点添加为任务节点

计算节点数量

计算节点数量则是自动确定的：所有不是任务节点的节点都可以作为计算节点：

见环境第267行

```
267     fog_node_ids = all_node_ids_set - task_node_ids
```